

Index No.: 230280L

Name: J. A. P. R. Jayasinghe

Github Link: https://github.com/pemitharj/EN3160_Assignment_1

EN3160 Assignment 1

Intensity Transformations and Neighborhood Filtering

```
In [15]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
```

Question 1

```
In [16]: def intensity_transform(im, breakpoints):
    breakpoints = np.array(breakpoints, dtype=np.float64)

    im_float = im.astype(np.float64)
    out = np.zeros_like(im_float)

    for i in range(len(breakpoints) - 1):
        x0, y0 = breakpoints[i]
        x1, y1 = breakpoints[i + 1]

        if x1 == x0:
            continue # vertical jump, no input range to map

        mask = (im_float >= x0) & (im_float < x1)
        slope = (y1 - y0) / (x1 - x0)
        out[mask] = y0 + slope * (im_float[mask] - x0)

    out[im_float == breakpoints[-1, 0]] = breakpoints[-1, 1]
    out = np.clip(out, 0, 255).astype(np.uint8)

    # Build a 256-point curve for plotting only
    all_x = np.arange(256, dtype=np.float64)
    curve = np.zeros_like(all_x)
    for i in range(len(breakpoints) - 1):
        x0, y0 = breakpoints[i]
        x1, y1 = breakpoints[i + 1]
        if x1 == x0:
            continue
        mask = (all_x >= x0) & (all_x < x1)
        slope = (y1 - y0) / (x1 - x0)
        curve[mask] = y0 + slope * (all_x[mask] - x0)
    curve[-1] = breakpoints[-1, 1]

    return out, curve
```

```
In [17]: img = cv.imread('Images/img1.jpeg', cv.IMREAD_GRAYSCALE)
breakpoints = [[0, 0], [50, 50], [50, 100], [100, 175], [150, 255], [150, 150], [25
```

```
transformed, curve = intensity_transform(img, breakpoints)
```

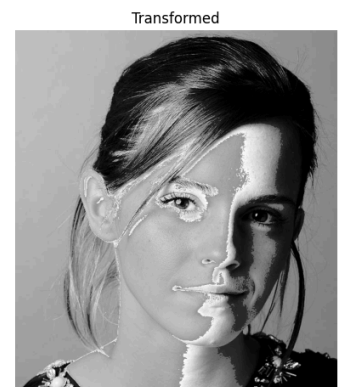
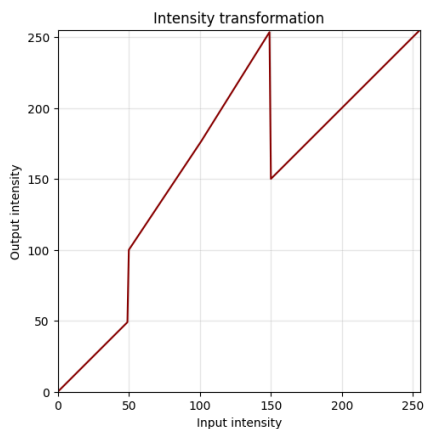
```
In [18]: # Plots
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].plot(np.arange(256), curve, color='darkred')
axes[0].set_xlim(0, 255)
axes[0].set_ylim(0, 255)
axes[0].set_xlabel('Input intensity')
axes[0].set_ylabel('Output intensity')
axes[0].set_title('Intensity transformation')
axes[0].grid(True, alpha=0.3)
axes[0].set_aspect('equal')

axes[1].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Original')
axes[1].axis('off')

axes[2].imshow(transformed, cmap='gray', vmin=0, vmax=255)
axes[2].set_title('Transformed')
axes[2].axis('off')

plt.tight_layout()
plt.show()
```



Interpretation and Discussion of Results

The grayscale image is transformed using a piecewise linear intensity transformation. The specified breakpoints determine how the input intensity values are mapped to output intensity values. Here, the middle intensity ranges are mapped to higher intensity ranges, making these regions appear brighter, while the higher and lower intensity ranges remain unchanged. Overall, the transformation modifies the brightness and contrast of the selected intensity regions. We can clearly see this in the resulting image, where Emma's face has changed in intensity compared with the original image.

Question 2

```
In [19]: brain = cv.imread('Images/img2.jpeg', cv.IMREAD_GRAYSCALE)

# Transform all white matter to maximum intensity.
```

```

white_matter_breakpoints = [[0, 0], [170, 0], [200, 255], [255, 255],]

# Tranform all gray matter to maximum intensity
gray_matter_breakpoints = [[0, 0], [100, 0], [130, 255], [160, 255], [200, 0], [255, 255]]

white_out, white_curve = intensity_transform(brain, white_matter_breakpoints)
gray_out, gray_curve = intensity_transform(brain, gray_matter_breakpoints)

```

```

In [20]: # White matter plots
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].plot(np.arange(256), white_curve, color='darkred')
axes[0].set_xlim(0, 255); axes[0].set_ylim(0, 255)
axes[0].set_xlabel('Input intensity'); axes[0].set_ylabel('Output intensity')
axes[0].set_title('White matter transformation')
axes[0].grid(True, alpha=0.3); axes[0].set_aspect('equal')

axes[1].imshow(brain, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Original'); axes[1].axis('off')

axes[2].imshow(white_out, cmap='gray', vmin=0, vmax=255)
axes[2].set_title('White matter accentuated'); axes[2].axis('off')

plt.tight_layout()
plt.show()

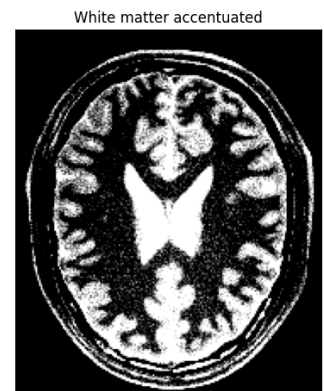
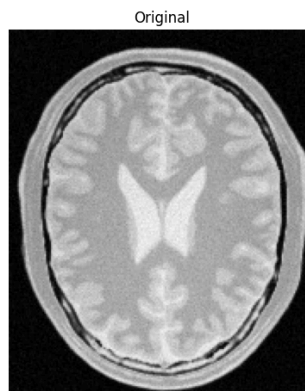
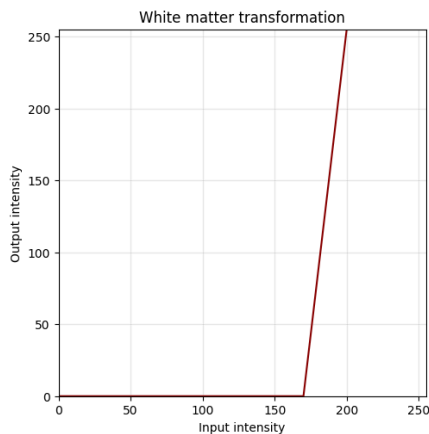
# Gray matter plots
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].plot(np.arange(256), gray_curve, color='darkred')
axes[0].set_xlim(0, 255); axes[0].set_ylim(0, 255)
axes[0].set_xlabel('Input intensity'); axes[0].set_ylabel('Output intensity')
axes[0].set_title('Gray matter transformation')
axes[0].grid(True, alpha=0.3); axes[0].set_aspect('equal')

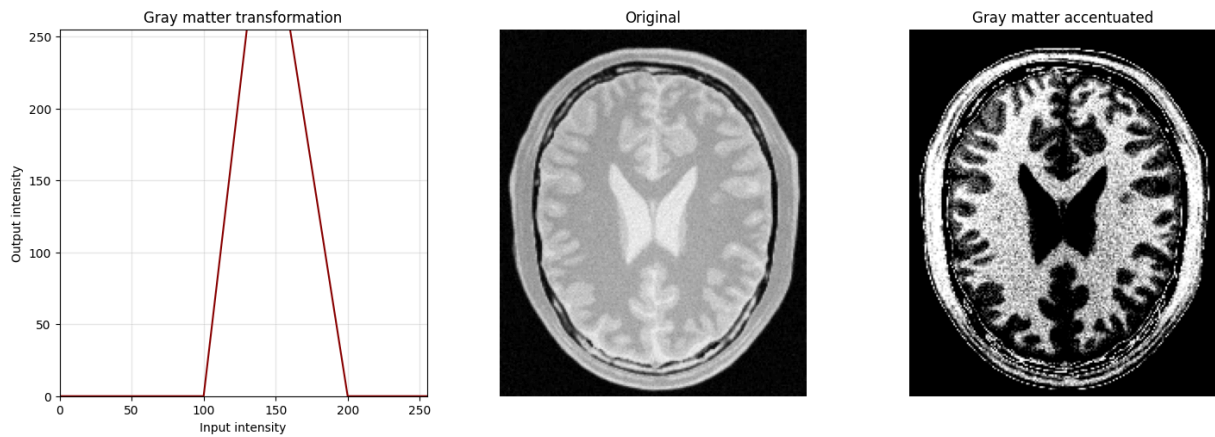
axes[1].imshow(brain, cmap='gray', vmin=0, vmax=255)
axes[1].set_title('Original'); axes[1].axis('off')

axes[2].imshow(gray_out, cmap='gray', vmin=0, vmax=255)
axes[2].set_title('Gray matter accentuated'); axes[2].axis('off')

plt.tight_layout()
plt.show()

```





Interpretation and Discussion of Results

For the white matter transformation, intensities above approximately 170 are mapped to higher intensities up to 255, since white matter generally appears at higher intensity levels. This makes the white matter brighter while the other regions remain dark. For the gray matter transformation, the approximate middle intensity range is mapped to 255, while the remaining intensity ranges are mapped to 0. Therefore, the gray matter appears brighter and the other regions remain dark. This shows that different intensity transformations can be used to accentuate different tissues in the brain image.

Question 3

```
In [21]: img_bgr = cv.imread('Images/img3.jpeg', cv.IMREAD_COLOR)
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

# Convert to L*a*b*
lab = cv.cvtColor(img_bgr, cv.COLOR_BGR2LAB)
L, a, b = cv.split(lab)
gamma = 0.6 # Apply gamma correction to the L plane

L_norm = L.astype(np.float64) / 255.0
L_gamma = np.power(L_norm, gamma) * 255.0
L_gamma = np.clip(L_gamma, 0, 255).astype(np.uint8)

lab_corrected = cv.merge([L_gamma, a, b])
img_corrected_bgr = cv.cvtColor(lab_corrected, cv.COLOR_LAB2BGR)
img_corrected_rgb = cv.cvtColor(img_corrected_bgr, cv.COLOR_BGR2RGB)
```

```
In [22]: # Original Image vs Gamma Corrected Image
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(img_rgb)
axes[0].set_title('Original')
axes[0].axis('off')

axes[1].imshow(img_corrected_rgb)
axes[1].set_title(f'Gamma corrected (γ = {gamma})')
axes[1].axis('off')

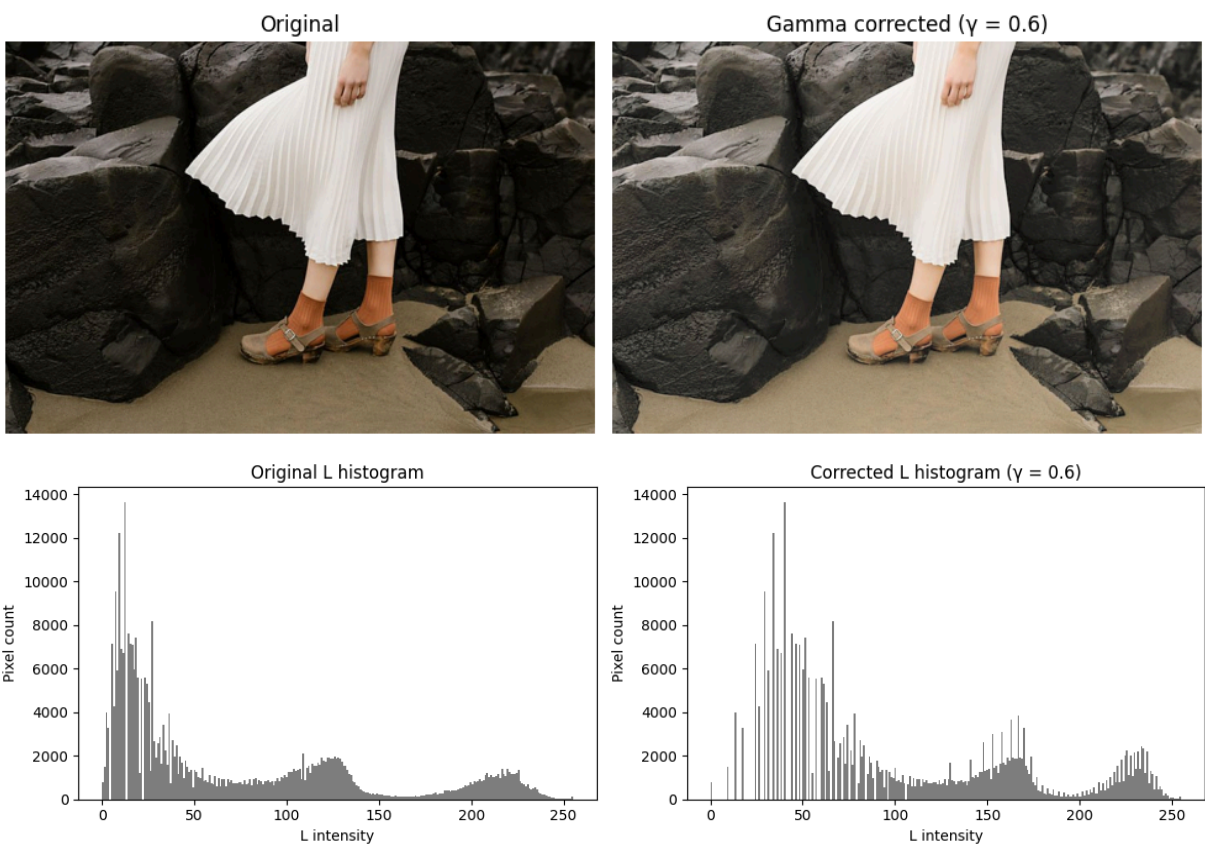
plt.tight_layout()
plt.show()
```

```
# Histograms of the L plane
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(L.ravel(), bins=256, range=(0, 255), color='gray')
axes[0].set_title('Original L histogram')
axes[0].set_xlabel('L intensity')
axes[0].set_ylabel('Pixel count')

axes[1].hist(L_gamma.ravel(), bins=256, range=(0, 255), color='gray')
axes[1].set_title(f'Corrected L histogram ( $\gamma = \{\text{gamma}\}$ ')
axes[1].set_xlabel('L intensity')
axes[1].set_ylabel('Pixel count')

plt.tight_layout()
plt.show()
```



Interpretation and Discussion of Results

The image is converted into the *Lab* colour space and split into the L, a, and b planes. Gamma correction is applied only to the L (lightness) plane with a gamma value of 0.6. Since gamma is less than 1, the L intensities are increased. This makes the corrected image appear brighter than the original image. The colour information in the a and b planes is preserved while the lightness is modified. Also, the corrected L histogram shows a change towards higher intensity values, which is consistent with the brighter appearance of the corrected image.

Question 4

```
In [23]: img_bgr = cv.imread('Images/img4.jpeg', cv.IMREAD_COLOR)
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

# (a) Split into hue, saturation, value planes
hsv = cv.cvtColor(img_bgr, cv.COLOR_BGR2HSV)
h, s, v = cv.split(hsv)

# (b) Vibrance transformation on the saturation plane
sigma = 70

def vibrance_transform(x, a, sigma=70):
    x = x.astype(np.float64)
    boosted = x + a * 128.0 * np.exp(-((x - 128.0) ** 2) / (2 * sigma ** 2))
    return np.clip(boosted, 0, 255)

# (c) Pick a
a = 0.6

s_boosted = vibrance_transform(s, a, sigma).astype(np.uint8)

# (d) Recombine the three planes
hsv_boosted = cv.merge([h, s_boosted, v])
img_boosted_bgr = cv.cvtColor(hsv_boosted, cv.COLOR_HSV2BGR)
img_boosted_rgb = cv.cvtColor(img_boosted_bgr, cv.COLOR_BGR2RGB)

# (e) Display original, vibrance-enhanced, and the transformation curve
x_vals = np.arange(256)
curve = vibrance_transform(x_vals, a, sigma)
```

```
In [24]: # Plots
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

axes[0].imshow(img_rgb)
axes[0].set_title('Original')
axes[0].axis('off')

axes[1].imshow(img_boosted_rgb)
axes[1].set_title(f'Vibrance enhanced (a = {a})')
axes[1].axis('off')

axes[2].plot(x_vals, curve, color='darkred')
axes[2].set_xlim(0, 255)
axes[2].set_ylim(0, 255)
axes[2].set_xlabel('Input saturation')
axes[2].set_ylabel('Output saturation')
axes[2].set_title('Saturation transformation')
axes[2].grid(True, alpha=0.3)
axes[2].set_aspect('equal')

plt.tight_layout()
plt.show()
```



Interpretation and Discussion of Results

The image is converted into the HSV colour space and split into the hue, saturation, and value planes. The vibrance transformation is applied only to the saturation plane. The parameter **a** is fine-tuned to obtain a visually pleasing result, and **a = 0.6** is selected. The transformation mainly increases the saturation of pixels around the middle intensity range. This makes the colours appear more vibrant. However, the hue and value planes are not changed, so the original hue and brightness information are preserved. Therefore, the resulting image has stronger colours without significantly changing its overall brightness. This can be clearly seen in the resulting image when compared with the original image.

Question 5

```
In [25]: f = cv.imread('Images/img5.tif', cv.IMREAD_GRAYSCALE)

M, N = f.shape
L = 256

hist, _ = np.histogram(f, bins=L, range=(0, L))
cdf = np.cumsum(hist)

x = np.array([round((L - 1) / (M * N) * cdf[k]) for k in range(L)], dtype=np.uint8)

y = x[f]
```

```
In [26]: # Original image vs Histogram equalized image
fig, ax = plt.subplots(1, 2, figsize=(12, 8))
ax[0].imshow(f, cmap='gray', vmin=0, vmax=255)
ax[0].set_title('Original Image')
ax[0].axis('off')

ax[1].imshow(y, cmap='gray', vmin=0, vmax=255)
ax[1].set_title('Histogram Equalized Image')
ax[1].axis('off')

plt.show()

# Histograms
hist_g, _ = np.histogram(y.ravel(), bins=L, range=(0, L))
```

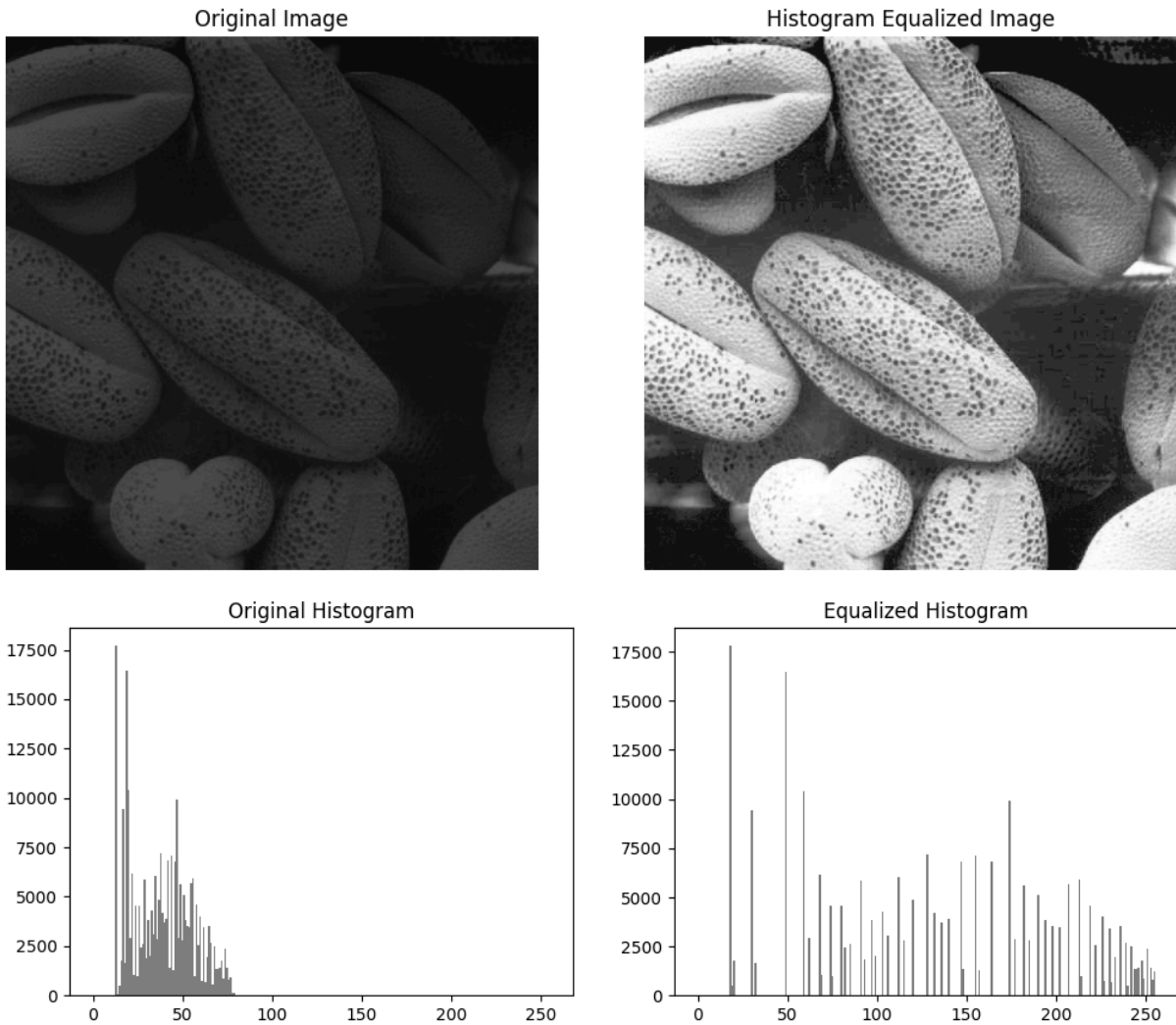
```

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].bar(np.arange(L), hist, color='gray', width=1)
ax[0].set_title('Original Histogram')

ax[1].bar(np.arange(L), hist_g, color='gray', width=1)
ax[1].set_title('Equalized Histogram')

plt.show()

```



Interpretation and Discussion of Results

The original image has most of its intensity values concentrated in a lower intensity range, so the image appears dark and has low contrast. After histogram equalization, the intensity values are spread over a wider range, so the image becomes brighter and the contrast is improved. Therefore, details in both the bright regions and dark regions become more visible. We can see the high peaks and gaps in the equalized histogram. These occur because the image has discrete intensity levels, so some output intensity levels contain few or no pixels. Overall, histogram equalization improves the visibility of details by expanding the original narrow intensity range.

Question 6

```

In [27]: # (a) Split into H, S, V
img = cv.imread('Images/img6.jpeg', cv.IMREAD_COLOR)
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
h, s, v = cv.split(hsv)

# (b) Select appropriate plane and threshold
_, mask = cv.threshold(s, 40, 255, cv.THRESH_BINARY)

# (c) Extract foreground and compute its histogram
foreground = cv.bitwise_and(img, img, mask=mask)
v_foreground = cv.bitwise_and(v, v, mask=mask)

# Histogram over foreground pixels only
fg_pixels = v[mask == 255]
hist, _ = np.histogram(fg_pixels, bins=256, range=(0, 256))

# (d) Cumulative sum
cdf = np.cumsum(hist)

# (e) Histogram equalize the foreground
L = 256
MN = fg_pixels.size

t = np.array([round((L - 1) / MN * cdf[k]) for k in range(L)], dtype=np.uint8)
v_eq_foreground = np.where(mask == 255, t[v], 0).astype(np.uint8)

# (f) Extract background and add with equalized foreground
background_mask = cv.bitwise_not(mask)
v_background = cv.bitwise_and(v, v, mask=background_mask)

v_result = cv.add(v_background, v_eq_foreground)

hsv_result = cv.merge([h, s, v_result])
result = cv.cvtColor(hsv_result, cv.COLOR_HSV2BGR)

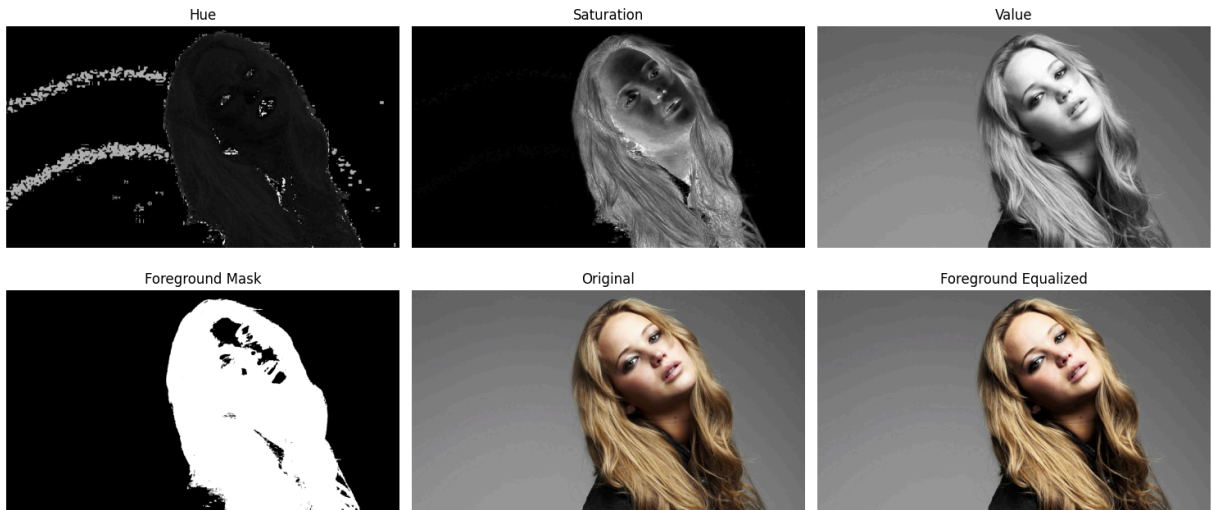
```

```

In [28]: # Plot H, S, V planes
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(h, cmap='gray'); ax[0].set_title('Hue'); ax[0].axis('off')
ax[1].imshow(s, cmap='gray'); ax[1].set_title('Saturation'); ax[1].axis('off')
ax[2].imshow(v, cmap='gray'); ax[2].set_title('Value'); ax[2].axis('off')
plt.tight_layout()
plt.show()

# Display mask, original, and output images
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(mask, cmap='gray'); ax[0].set_title('Foreground Mask'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB)); ax[1].set_title('Original'); ax[1].axis('off')
ax[2].imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB)); ax[2].set_title('Foreground Eq'); ax[2].axis('off')
plt.tight_layout()
plt.show()

```



Interpretation and Discussion of Results

The image is converted into the HSV colour space and separated into the H, S, and V planes. The saturation plane is selected because the foreground and background have a clear difference in saturation, making it suitable for separating them. A threshold of 40 is selected to obtain a suitable foreground mask, where pixels with saturation above 40 are considered foreground. The mask is then used to extract the foreground pixels from the V plane. Histogram equalization is applied only to the foreground V intensities, while the background is kept unchanged. The equalized foreground and original background are then combined to obtain the final image. This improves the contrast of the foreground while preserving the original background. We can clearly see this in the resulting image, where Jennifer is more emphasized against the background compared with the original image.

Question 7

```
In [29]: img = cv.imread('Images/img7.jpeg', cv.IMREAD_GRAYSCALE)

# (a) Using cvfilter2D to sobel horizontal filter
sobel_x = np.array([[1, 0, -1], [2, 0, -2], [1, 0, -1]])

img_x = cv.filter2D(img, cv.CV_64F, sobel_x)

# (b) Manual Sobel filtering
def sobel_filter(img, kernel):
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2

    # Reflect the image at the borders
    img_padded = np.pad(img.astype(np.float64), ((pad_h, pad_h), (pad_w, pad_w)), m

    output = np.zeros_like(img, dtype=np.float64)

    # Apply the Sobel kernel manually
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            region = img_padded[i:i + kh, j:j + kw]
            output[i, j] = np.sum(region * kernel)
```

```

    return output

img_x_manual = sobel_filter(img, sobel_x)

# (c) Sobel filtering using separability
col_kernel = np.array([[1], [2], [1]], dtype=np.float64)
row_kernel = np.array([[1, 0, -1]], dtype=np.float64)

temp = cv.filter2D(img, cv.CV_64F, col_kernel)
img_x_separable = cv.filter2D(temp, cv.CV_64F, row_kernel)

```

```

In [30]: # Output images
fig, ax = plt.subplots(1, 3, figsize=(18, 6))

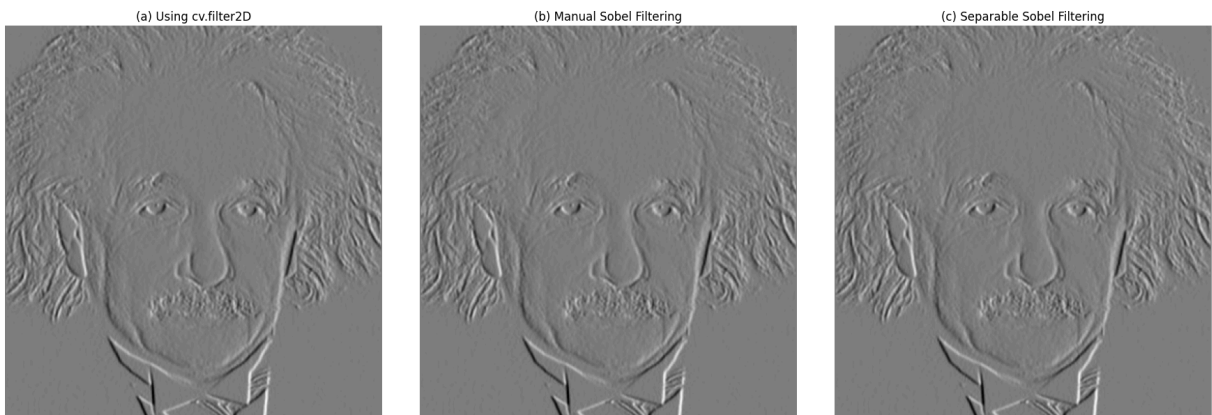
ax[0].imshow(img_x, cmap='gray')
ax[0].set_title('(a) Using cv.filter2D')
ax[0].axis('off')

ax[1].imshow(img_x_manual, cmap='gray')
ax[1].set_title('(b) Manual Sobel Filtering')
ax[1].axis('off')

ax[2].imshow(img_x_separable, cmap='gray')
ax[2].set_title('(c) Separable Sobel Filtering')
ax[2].axis('off')

plt.tight_layout()
plt.show()

```



Interpretation and Discussion of Results

The grayscale image is filtered using the horizontal Sobel kernel to detect vertical edges. First, the Sobel filter is applied using `cv.filter2D`. The same filtering operation is then implemented manually using the kernel and pixel neighbourhoods. Finally, the Sobel kernel is separated into a column kernel and a row kernel, and filtering is performed in two steps. The three output images show similar edge patterns, demonstrating that the manual and separable implementations produce results comparable to `cv.filter2D`. Therefore, the results demonstrate that the Sobel operator can be implemented in different ways while producing similar edge information.

Question 8

```
In [31]: # Define function for zooming images using nearest neighbor and bilinear methods
def zoom_image(im, s, method='nearest'):
    im = im.astype(np.float64)
    H, W, _ = im.shape
    new_H, new_W = round(H * s), round(W * s)
    ys = (np.arange(new_H) + 0.5) / s - 0.5
    xs = (np.arange(new_W) + 0.5) / s - 0.5

    if method == 'nearest':
        yi = np.clip(np.round(ys), 0, H - 1).astype(int)
        xi = np.clip(np.round(xs), 0, W - 1).astype(int)
        out = im[yi][:, xi]
    else: # bilinear
        y0 = np.clip(np.floor(ys), 0, H - 1).astype(int); y1 = np.clip(y0 + 1, 0, H
        x0 = np.clip(np.floor(xs), 0, W - 1).astype(int); x1 = np.clip(x0 + 1, 0, W
        wy = (ys - y0)[:, None, None]; wx = (xs - x0)[None, :, None]
        Ia, Ib, Ic, Id = im[y0][:, x0], im[y0][:, x1], im[y1][:, x0], im[y1][:, x1]
        out = (Ia * (1 - wx) + Ib * wx) * (1 - wy) + (Ic * (1 - wx) + Id * wx) * wy

    return np.clip(out, 0, 255).astype(np.uint8)

# Define function to calculate normalized SSD
def ssd(a, b):
    return np.sum((a.astype(np.float64) - b.astype(np.float64)) ** 2) / a.size
```

```
In [32]: pairs = [('Images/img10small.png', 'Images/img10.png'), ('Images/img11small.png', 'I

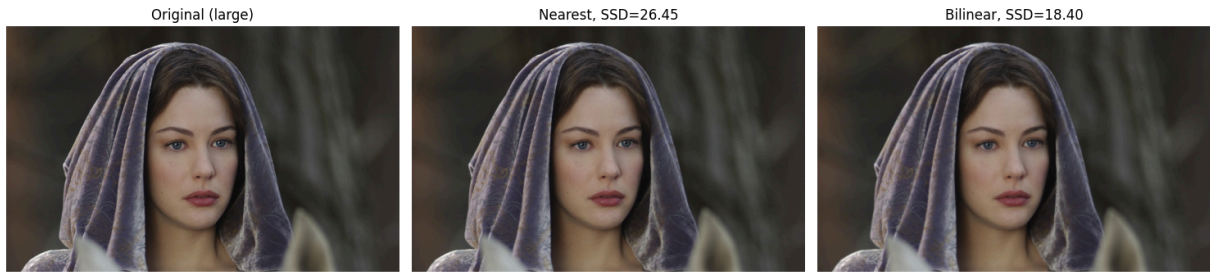
for small_path, large_path in pairs:
    small = cv.cvtColor(cv.imread(small_path, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)
    original = cv.cvtColor(cv.imread(large_path, cv.IMREAD_COLOR), cv.COLOR_BGR2RGB)

    zoomed_nn = zoom_image(small, 4, 'nearest')
    zoomed_bl = zoom_image(small, 4, 'bilinear')

    h, w = min(zoomed_nn.shape[0], original.shape[0]), min(zoomed_nn.shape[1], orig
    ssd_nn = ssd(zoomed_nn[:h, :w], original[:h, :w])
    ssd_bl = ssd(zoomed_bl[:h, :w], original[:h, :w])

    fig, ax = plt.subplots(1, 3, figsize=(15, 5))
    ax[0].imshow(original); ax[0].set_title('Original (large)'); ax[0].axis('off')
    ax[1].imshow(zoomed_nn); ax[1].set_title(f'Nearest, SSD={ssd_nn:.2f}'); ax[1].a
    ax[2].imshow(zoomed_bl); ax[2].set_title(f'Bilinear, SSD={ssd_bl:.2f}'); ax[2].
    plt.tight_layout()
    plt.show()
```





Interpretation and Discussion

The small images are enlarged by a scale factor of 4 using nearest-neighbour and bilinear interpolation. The enlarged images are compared with the corresponding original large images using the normalized SSD.

For **img10** (top image), the nearest-neighbour method gives an SSD of **136.27**, while bilinear interpolation gives a lower SSD of **115.12**. For **img11** (bottom image), the nearest-neighbour method gives an SSD of **26.45**, while bilinear interpolation gives a lower SSD of **18.40**. Since a lower SSD indicates a smaller difference from the original image, bilinear interpolation produces results closer to the original large images for both cases. This is because bilinear interpolation uses neighbouring pixels to produce smoother intensity transitions, whereas nearest-neighbour interpolation can produce more blocky or pixelated results.

Question 9

```
In [33]: img = cv.imread('Images/img8.jpeg', cv.IMREAD_COLOR)

mask = np.zeros(img.shape[:2], np.uint8)
mask[:] = cv.GC_PR_BGD # Set the outer region as probable background
rect = (50, 20, img.shape[1] - 100, img.shape[0] - 40) # Rectangle containing the f
x, y, w, h = rect
mask[y:y+h, x:x+w] = cv.GC_PR_FGD
cv.circle(mask, (300, 100), 40, cv.GC_BGD, -1)

# (a) Using GrabCut
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
cv.grabCut(img, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_MASK)

# Extract foreground
mask2 = np.where((mask == cv.GC_FGD) | (mask == cv.GC_PR_FGD), 1, 0).astype(np.uint
foreground = img * mask2[:, :, np.newaxis]

# Extract background
background = cv.subtract(img, foreground)

# (b) Enhanced image
mask_3ch = cv.merge([mask2] * 3)
background_only = (img * (1 - mask_3ch)).astype(np.uint8)
blurred_full = cv.GaussianBlur(background_only, (35, 35), 0)
```

```
enhanced = (img * mask_3ch + blurred_full * (1 - mask_3ch)).astype(np.uint8)
```

```
In [34]: # Outputs
fig, ax = plt.subplots(1, 5, figsize=(25, 5))

ax[0].imshow(mask2, cmap='gray')
ax[0].set_title('Segmentation Mask')
ax[0].axis('off')

ax[1].imshow(cv.cvtColor(foreground, cv.COLOR_BGR2RGB))
ax[1].set_title('Foreground Image')
ax[1].axis('off')

ax[2].imshow(cv.cvtColor(background, cv.COLOR_BGR2RGB))
ax[2].set_title('Background Image')
ax[2].axis('off')

ax[3].imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
ax[3].set_title('Original Image')
ax[3].axis('off')

ax[4].imshow(cv.cvtColor(enhanced, cv.COLOR_BGR2RGB))
ax[4].set_title('Enhanced Image')
ax[4].axis('off')

plt.tight_layout()
plt.show()
```



Interpretation and Discussion of Results

GrabCut is used to segment the flower from the background using the defined mask and rectangle. The resulting segmentation mask is used to extract the foreground and background separately. The background is then blurred using Gaussian filtering, while the segmented foreground is kept unchanged. These are combined to produce the enhanced image, where the flower remains clear and the background becomes blurred.

Why is the background just beyond the edge of the flower quite dark in the enhanced image?

The background just beyond the edge of the flower becomes quite dark because the foreground pixels are set to zero before the background is blurred. Gaussian filtering calculates each pixel value using a weighted average of neighbouring pixels. Therefore, pixels near the flower boundary include some zero-valued foreground pixels in the averaging process, which reduces their intensity. As the distance from the flower increases,

the effect of the zero-valued pixels becomes smaller. Therefore, the dark region appears mainly near the edge of the flower.

Question 10

```
In [35]: img = cv.imread('Images/img9.jpeg', cv.IMREAD_COLOR)
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)

# (a) OpenCV's bilateral filter vs Gaussian blur
sigma_s, sigma_r = 15, 75

bilateral_cv = cv.bilateralFilter(img, d=9, sigmaColor=sigma_r, sigmaSpace=sigma_s)
gaussian = cv.GaussianBlur(img, (9, 9), sigma_s)

# (b) Own bilateral filter implementation
def bilateral_filter_manual(im, d, sigma_s, sigma_r):
    im = im.astype(np.float64)
    H, W, C = im.shape
    pad = d // 2
    im_padded = np.pad(im, ((pad, pad), (pad, pad), (0, 0)), mode='reflect')
    out = np.zeros_like(im)

    # Precompute spatial Gaussian weights
    ax_range = np.arange(-pad, pad + 1)
    gx, gy = np.meshgrid(ax_range, ax_range)
    spatial_weight = np.exp(-(gx**2 + gy**2) / (2 * sigma_s**2))

    for i in range(H):
        for j in range(W):
            region = im_padded[i:i + d, j:j + d, :]
            center = im[i, j, :]
            range_weight = np.exp(-np.sum((region - center) ** 2, axis=2) / (2 * sigma_r**2))
            weight = spatial_weight * range_weight
            weight_sum = np.sum(weight)
            out[i, j, :] = np.sum(region * weight[:, :, np.newaxis], axis=(0, 1)) / weight_sum

    return np.clip(out, 0, 255).astype(np.uint8)

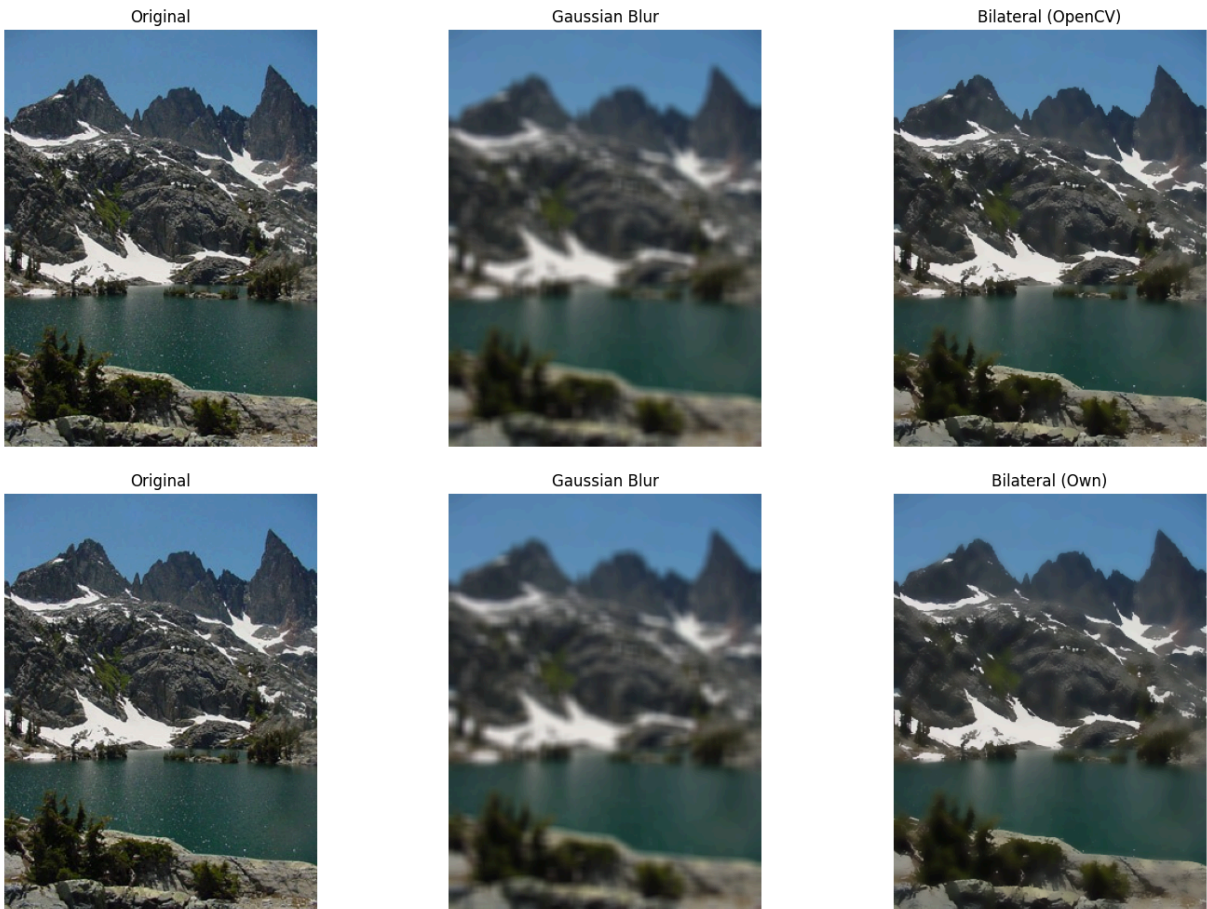
bilateral_manual = bilateral_filter_manual(img, d=9, sigma_s=sigma_s, sigma_r=sigma_r)
```

```
In [36]: # Gaussian blur and OpenCv's bilateral filter images
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(img_rgb); ax[0].set_title('Original'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(gaussian, cv.COLOR_BGR2RGB)); ax[1].set_title('Gaussian Bl')
ax[2].imshow(cv.cvtColor(bilateral_cv, cv.COLOR_BGR2RGB)); ax[2].set_title('Bilater')
plt.tight_layout()
plt.show()

# Gaussian blur and own bilateral filter images
fig, ax = plt.subplots(1, 3, figsize=(15, 5))
ax[0].imshow(img_rgb); ax[0].set_title('Original'); ax[0].axis('off')
ax[1].imshow(cv.cvtColor(gaussian, cv.COLOR_BGR2RGB)); ax[1].set_title('Gaussian Bl')
ax[2].imshow(cv.cvtColor(bilateral_manual, cv.COLOR_BGR2RGB)); ax[2].set_title('Bil')
plt.tight_layout()
```

```
plt.show()
```

```
# Quantitative comparison: Own bilateral vs OpenCV's bilateral result
mse = np.mean((bilateral_cv.astype(np.float64) - bilateral_manual.astype(np.float64)
psnr = cv.PSNR(bilateral_cv, bilateral_manual)
print(f'MSE (OpenCV vs Own): {mse:.4f}')
print(f'PSNR (OpenCV vs Own): {psnr:.2f} dB')
```



MSE (OpenCV vs Own): 28.7867
PSNR (OpenCV vs Own): 33.54 dB

Interpretation and Discussion of Results

The original image is filtered using Gaussian blur and bilateral filtering. Gaussian blur smooths the image but also blurs the edges, while bilateral filtering reduces noise while preserving the edges of the image. Therefore, the bilateral filtered image retains sharper edges compared with the Gaussian-blurred image.

Comparison between my own bilateral filter vs OpenCV's bilateral filter

I also implemented the bilateral filter using spatial and intensity based weights. The result from my own implementation is compared with OpenCV's bilateral filter. The MSE between the two results is **28.79**, while the PSNR is **33.54 dB**. The relatively low MSE and high PSNR indicate that my own implemented filter produces a result that is quite similar to OpenCV's implementation. The small difference can be due to differences in implementation details and numerical calculations.